

102. 二叉树的层序遍历

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 广度优先搜索, 二叉树
- **Companies:** (Not Available)

给你二叉树的根节点 `root`，返回其节点值的 **层序遍历**。（即逐层地，从左到右访问所有节点）。

示例 1:

输入：root = [3,9,20,null,null,15,7]

输出：[[3],[9,20],[15,7]]

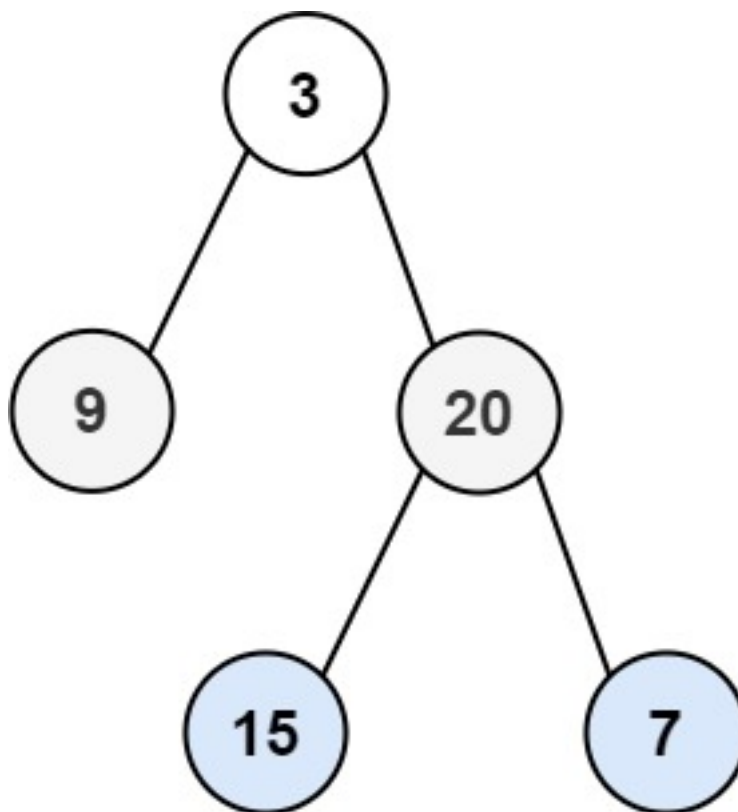


Figure 1: img

示例 2:

输入：root = [1]

输出：[[1]]

示例 3:

输入：root = []

输出：[]

提示:

- 树中节点数目在范围 [0, 2000] 内
- $-1000 \leq \text{Node.val} \leq 1000$

Discussion | Solution

解决方法

1. 问题理解

目标: 对给定的二叉树进行层序遍历, 并返回一个包含每层节点值的列表的列表。**层序遍历:** 从上到下, 从左到右, 逐层访问节点。

2. 思路分析

层序遍历的典型实现方法是使用**广度优先搜索 (BFS)**, 借助队列来完成。

方法一: 迭代 (广度优先搜索 - BFS)

1. 初始化:

- 创建一个空列表 result 用于存储最终结果 (每层的节点列表)。
- 如果 root 为 null, 直接返回 result。
- 创建一个队列 queue (通常使用 collections.deque), 并将根节点 root 入队。

2. 层序遍历循环:

- 当队列 queue 不为空时, 进行循环 (每次循环处理一层):
 - 获取当前层的节点数量 level_size = len(queue)。这个值非常关键, 它确定了当前层有多少个节点需要处理。
 - 创建一个空列表 current_level_nodes 用于存储当前层所有节点的值。
 - **内层循环**, 执行 level_size 次, 处理当前层的所有节点:
 - * 将队首节点 node 出队 (queue.popleft())。
 - * 将 node.val 添加到 current_level_nodes 列表中。
 - * 如果 node 的左子节点不为空, 将其入队 (queue.append(node.left))。
 - * 如果 node 的右子节点不为空, 将其入队 (queue.append(node.right))。
 - 内层循环结束后, current_level_nodes 就包含了当前层所有节点的值。将其添加到 result 列表中 (result.append(current_level_nodes))。

- 3. **返回结果:** 外层循环结束后, result 就包含了所有层的节点值列表, 返回 result。

执行流程示例 ([3,9,20,null,null,15,7]):

步骤	queue	level_size	current_level	result	操作
1	[3]	-	-	[]	初始化
2	[3]	1	[]	[]	开始处理第 1 层
3	[]	1	[3]	[]	出队 3, 加入 current_level
4	[9, 20]	1	[3]	[]	入队 9, 20
5	[9, 20]	1	[3]	[[3]]	第 1 层处理完毕, 加入 result
6	[9, 20]	2	[]	[[3]]	开始处理第 2 层
7	[20]	2	[9]	[[3]]	出队 9, 加入 current_level (无子节点)
8	[15, 7]	2	[9, 20]	[[3]]	出队 20, 加入 current_level, 入队 15, 7
9	[15, 7]	2	[9, 20]	[[3], [9, 20]]	第 2 层处理完毕, 加入 result
10	[15, 7]	2	[]	[[3], [9, 20]]	开始处理第 3 层
11	[7]	2	[15]	[[3], [9, 20]]	出队 15, 加入 current_level (无子节点)
12	[]	2	[15, 7]	[[3], [9, 20]]	出队 7, 加入 current_level (无子节点)
13	[]	2	[15, 7]	[[3], [9, 20], [15, 7]]	第 3 层处理完毕, 加入 result
14	[]	-	-	[[3], [9, 20], [15, 7]]	队列为空, 循环结束, 返回 result

方法二：递归（深度优先搜索 - DFS） 虽然 BFS 是层序遍历最自然的方法，但也可以使用 DFS 实现。关键在于递归时需要传递当前的层级信息。

1. 初始化:

- 创建一个空列表 result 用于存储最终结果。

2. 定义递归函数 dfs(node, level):

- **功能:** 访问 node 节点, 并将其值添加到 result 中对应层级的列表里。
- **基本情况 (Base Case):** 如果 node 为 null, 直接返回。
- **处理当前层:**

- 检查 result 的长度是否小于 level + 1。如果是, 说明这是第一次访问到 level 这一层, 需要在 result 中添加一个新的空列表来存储这一层的节点: result.append([])。
 - 将 node.val 添加到 result[level] 这个列表中。
 - 递归步骤 (Recursive Step):
 - 递归调用左子节点: dfs(node.left, level + 1)。
 - 递归调用右子节点: dfs(node.right, level + 1)。
3. 主函数调用:
- 如果 root 不为 null, 调用 dfs(root, 0) 开始递归。
 - 返回 result。

注意: DFS 访问节点的顺序不是层序的, 但通过传递 level 参数, 可以确保节点值被正确地放入对应层级的列表中。

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import List, Optional
import collections

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # 方法一: 迭代 (BFS)
    def levelOrderBFS(self, root: Optional[TreeNode]) -> List[List[int]]:
        """
        使用迭代 (BFS) 实现层序遍历
        """
        result = []
        if not root:
            return result

        queue = collections.deque([root])

        while queue:
            level_size = len(queue)
```

```

        current_level_nodes = []
        for _ in range(level_size):
            node = queue.popleft()
            current_level_nodes.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(current_level_nodes)

    return result

# 方法二: 递归 (DFS)
def levelOrderDFS(self, root: Optional[TreeNode]) -> List[List[int]]:
    """
    使用递归 (DFS) 实现层序遍历
    """
    result = []

    def dfs(node: Optional[TreeNode], level: int):
        if not node:
            return

        # 如果当前层数超出了 result 的长度, 说明需要为新层创建列表
        if level >= len(result):
            result.append([])

        # 将当前节点值加入对应层的列表
        result[level].append(node.val)

        # 递归处理左右子节点
        dfs(node.left, level + 1)
        dfs(node.right, level + 1)

    dfs(root, 0)
    return result

# 最终提交可以选择其中一种方法
def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
    return self.levelOrderBFS(root)
    # return self.levelOrderDFS(root)

```

4. 复杂度分析

- 迭代方法 (BFS):
 - 时间复杂度: $O(N)$, 其中 N 是树中的节点数。每个节点入队出队各一次。

- 空间复杂度: $O(W)$, 其中 W 是树的最大宽度。队列中最多存储一层节点, 最坏情况下 (完全二叉树的最后一层) 可能达到 $O(N)$ 。

- **递归方法 (DFS):**

- 时间复杂度: $O(N)$ 。每个节点被访问一次。
- 空间复杂度: $O(H)$, 其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下 (链状树) 为 $O(N)$, 平均情况下 (平衡树) 为 $O(\log N)$ 。result 列表的空间复杂度为 $O(N)$ 。

5. 如何在面试中讲解

1. 明确目标:

- “目标是实现二叉树的层序遍历, 即按层从上到下、从左到右访问节点, 并将每层节点的值存入一个列表, 最终返回这些列表组成的列表。”

2. 迭代思路 (BFS):

- “层序遍历最自然的方法是 BFS。使用一个队列。”
- “核心技巧: 在处理每一层之前, 先记录下当前队列的大小 level_size。然后循环 level_size 次, 将这些节点出队、处理 (记录值), 并将其下一层的子节点入队。”
- “每次外层循环处理完一层, 将该层节点值的列表加入最终结果列表。”
- “分析复杂度: 时间 $O(N)$, 空间 $O(W)$ 。”

3. 递归思路 (DFS):

- “虽然不那么直观, 但也可以用 DFS 实现。关键是递归时需要传递当前的层级 level。”
- “定义 dfs(node, level)。Base Case 是 node 为空。”
- “在处理 node 时, 检查 result 列表是否已经有 level 对应的子列表。如果没有, 就创建一个。然后将 node.val 加入 result[level]。”
- “之后递归调用 dfs(node.left, level + 1) 和 dfs(node.right, level + 1)。”
- “分析复杂度: 时间 $O(N)$, 空间 $O(H)$ (递归栈) + $O(N)$ (结果列表)。”

4. 对比与选择:

- “BFS 是层序遍历的标准且更直观的方法。”
- “DFS 需要额外传递层级信息, 并且节点访问顺序不是层序的, 但也能得到正确结果。”

5. 边界条件:

- “处理空树 (root 为 null) 的情况, 返回空列表。”